

گزارش جامع و نهایی پروژه سامانه آنلاین رزرو و خرید بلیت مسابقات ورزشی

پلتفرم BookingSportTickets - از طراحی مفهومی پایگاه داده تا پیاده سازی سرویس بک اند، موتور جستجو و رابط کاربری

تیم توسعه: ۳ دانشجوی پر تلاش

پروژه: سامانه جامع بلیت فروشی ورزشی

میزبانی دیتابیس: PostgreSQL

(Neon.tech)

تاریخ تنظیم: مرداد ۱۴۰۵

فهرست مطالب گزارش

۱. خلاصه مدیریتی و اهداف پروژه

۲. فاز اول: تحلیل و طراحی ۳۰ جدول پایگاه داده (3NF)

۳. فاز دوم: بهینه سازی دیتابیس، پروسجرها و ایندکس گذاری

۴. فاز سوم: توسعه سرویس بک اند (FastAPI)، Redis و امنیت

۵. فاز چهارم: رابط کاربری (Frontend) و Elasticsearch

۶. تحلیل جریان های کاری کلیدی (End-to-End Workflows)

۷. ارزیابی نهایی و دستاوردهای پروژه

۱. خلاصه مدیریتی و اهداف پروژه

پروژه **BookingSportTickets** با هدف ارائه یک پلتفرم جامع، مقیاس‌پذیر و فوق‌العاده سریع برای خرید و رزرو آنلاین بلیت مسابقات مختلف ورزشی (فوتبال، والیبال، بسکتبال و...) طراحی و پیاده‌سازی شده است. این سیستم به گونه‌ای معماری شده که توانایی مدیریت تراکنش‌های مالی همزمان، جلوگیری از فروش تکراری بلیت (Overbooking)، جستجوی چندمعیاره با سرعت میلی‌ثانیه‌ای و مدیریت هوشمند قوانین استرداد و پشتیبانی را دارا باشد.

توسعه این نرم‌افزار طی چهار فاز منسجم و تکاملی انجام پذیرفته است:

فاز دوم: منطق پایگاه داده

توسعه Stored Procedureها، توابع اختصاصی، کوئری‌های پیچیده گزارش‌گیری و ایندکس‌گذاری B-Tree جهت افزایش بازدهی PostgreSQL.

فاز اول: تحلیل و نرم‌سازی

تحلیل ۳۰ جدول پایگاه داده بر اساس استاندارد 3NF، تفکیک موجودیت‌ها و پیاده‌سازی چندریختی (Polymorphism) برای ورزش‌های مختلف [cite: 4].

فاز چهارم: فرانت‌اند و موتور

جستجو

توسعه رابط کاربری چابک با Vanilla JS، ماژول شبکه متمرکز `api.js` و یکپارچه‌سازی Elasticsearch 8.x برای فیلتر آنی [cite: 6].

فاز سوم: سرویس‌دهی بک‌اند

پیاده‌سازی REST APIها با FastAPI، مدیریت قفل‌های موقت و کش با Redis (Upstash) و ساختار احراز هویت JWT/RBAC [cite: 5].

۲. فاز اول: تحلیل و طراحی ۳۰ جدول پایگاه داده

(3NF)

پایگاه داده سامانه با ۳۰ جدول کاملاً نرمال‌سازی‌شده (فرم نرمال سوم - 3NF) طراحی شده است [cite: 4]. این ساختار از وجود داده‌های تکراری و ناهنجاری‌های ویرایشی جلوگیری کرده و انعطاف‌پذیری بالایی را فراهم می‌سازد [cite: 4]. موجودیت‌های سیستم به ۷ حوزه اصلی تقسیم شده‌اند [cite: 4]:

۲.۱. مدیریت کاربران، دسترسی‌ها و کیف پول

- **ROLES (نقش‌ها):** تعیین سطح دسترسی افراد (تماشاگر عادی، پشتیبان، مدیر) جهت کنترل دسترسی متمرکز و رعایت 3NF [cite: 4].
- **USERS (کاربران):** نگهداری اطلاعات پایه احراز هویت (نام، ایمیل، تلفن، هش رمز عبور) [cite: 4].
- **USERS_SUPPORT (کاربران پشتیبانی):** نگهداری اطلاعات اختصاصی کادر پشتیبانی (کد پرسنلی) با رابطه 1:1 جهت جلوگیری از ستون‌های NULL در جدول اصلی [cite: 4].
- **WALLETS (کیف پول):** موجودی اعتباری حساب کاربر جهت پرداخت سریع و تسریع فرآیند استرداد وجه [cite: 4].
- **TRANSACTIONS_WALLET (تراکنش‌های کیف پول):** ثبت دقیق گردش مالی (شارژ، برداشت، عودت) جهت حسابرسی مالی کامل (Audit Trail) [cite: 4].

۲.۲. تقسیمات کشوری و مکان‌ها

- **PROVINCES (استان‌ها) & CITIES (شهرها):** ساختار هرمی برای جلوگیری از خطای تاپپی و فیلتر مسابقات بر اساس موقعیت جغرافیایی [cite: 4].
- **VENUES (ورزشگاه‌ها/سالن‌ها):** اطلاعات محل برگزاری (نام، آدرس و ظرفیت کل) جهت کنترل عدم فروش مازاد بلیت [cite: 4].

۲.۳. ساختار ورزشی، مسابقات و برگزارکنندگان

- **TYPES_SPORT (انواع رشته‌های ورزشی):** تعریف ورزش‌های مختلف (فوتبال، والیبال، بسکتبال) [cite: 4].
- **ORGANIZERS (برگزارکنندگان):** نهادهای مسئول (سازمان لیگ، فدراسیون‌ها) جهت تعیین سیاست‌های کنسلی مجزا [cite: 4].
- **TEAMS (تیم‌ها):** تعریف تیم‌های ورزشی، شهر و ورزش مرجع جهت تعیین تیم میزبان/میزمان [cite: 4].

• **COMPETITIONS (تورنمنت‌ها/لیگ‌ها):** دسته‌بندی مسابقات بر اساس فصل و

تورنمنت (لیگ برتر، جام حذفی) [cite: 4].

• **MATCHES (مسابقات):** هسته اصلی رویدادها شامل ترکیب "تیم میزبان + میهمان +

ورزشگاه + زمان + برگزارکننده" [cite: 4].

۲.۴. بلیت‌فروشی، صندلی‌ها و جزئیات اختصاصی (Polymorphism)

• **CATEGORIES_TICKET (دسته‌بندی بلیت‌ها):** بخش‌بندی استادیوم (روبروی جایگاه،

جایگاه ویژه، پشت دروازه) برای قیمت‌گذاری متفاوت [cite: 4].

• **TICKETS (بلیت‌ها):** بسته‌های بلیت تعریف‌شده برای یک مسابقه با قیمت مشخص و

ظرفیت باقیمانده [cite: 4].

• **SEATS (صندلی‌ها/جایگاه‌ها):** شماره دقیق صندلی و ردیف جهت امکان رزرو صندلی

مشخص و جلوگیری از فروش تکراری [cite: 4].

• **FACILITIES (امکانات جانبی) & FACILITIES_TICKET:** مدیریت امکانات جانبی

(پارکینگ، پذیرایی) و شکستن رابطه چندبه‌چند (M:N) برای رعایت 1NF [cite: 4].

• **DETAILS_FOOTBALL / VOLLEYBALL / BASKETBALL:** پیاده‌سازی الگوی

Polymorphism در 3NF؛ جداسازی ویژگی‌های خاص هر ورزش (مانند شماره گیت فوتبال یا

فودکورت بسکتبال) جهت پر نشدن جدول اصلی بلیت با ستون‌های NULL [cite: 4].

۲.۵. فرآیند رزرو، پرداخت، کنسلی و پشتیبانی

• **RESERVATIONS (رزروها):** درخواست اولیه خرید شامل تعداد بلیت، مبلغ کل، وضعیت و

مهلت رزرو موقت (reserved_until) [cite: 4].

• **SEATS_RESERVED (صندلی‌های قفل‌شده):** مدیریت همزمانی (Concurrency

Control) جهت جلوگیری از انتخاب همزمان یک صندلی توسط دو کاربر [cite: 4].

• **PAYMENTS (پرداخت‌ها):** ثبت رسمی کدهای پیگیری، زمان و وضعیت پرداخت

درگاه [cite: 4].

• **POLICIES_CANCELLATION & RULES_POLICY_CANCELLATION:** تعاریف قوانین

استرداد و درصدهای جریمه بر اساس بازه زمانی مانده تا بازی [cite: 4].

• **REQUESTS_CANCELLATION & REFUNDS:** ثبت درخواست کنسلی، بررسی

پشتیبان و صدور سند عودت وجه به کیف پول [cite: 4].

۳. فاز دوم: بهینه‌سازی دیتابیس، پروسجرها و ایندکس‌گذاری

در فاز دوم، تمرکز اصلی بر انتقال منطق‌های پیچیده دیتابیس به داخل PostgreSQL، افزایش سرعت اجرای کوئری‌ها و اطمینان از صحت داده‌ها در محیط ابری Neon.tech بوده است.

۳.۱. توابع و Stored Procedureها

به منظور افزایش امنیت و جلوگیری از ارسال کوئری‌های خام سنگین از سمت بک‌اند، پروسجرهای کلیدی پیاده‌سازی شدند:

- **sp_get_purchased_tickets_by_city**: بازیابی سوابق بلیت‌های خریداری‌شده توسط کاربر به همراه تفکیک شهر و اطلاعات کامل ورزشگاه.
- **پروسجرهای محاسبه جریمه کنسلی**: محاسبه هوشمند میزان جریمه بر اساس زمان فعلی و جدول **RULES_POLICY_CANCELLATION**.
- **پروسجر نهایی‌سازی پرداخت**: ثبت همزمان پرداخت، تغییر وضعیت صندلی و بروزرسانی کیف پول در قالب یک Transaction یکپارچه.

۳.۲. استراتژی ایندکس‌گذاری (Indexing)

برای بهینه‌سازی کوئری‌های پرکاربرد، ایندکس‌های زیر روی دیتابیس PostgreSQL اعمال گردید:

- **B-Tree Composite Index روی جدول MATCHES**: ترکیب ستون‌های **(start_time, venue_id)** جهت سرعت‌بخشی به لیست‌کردن بازی‌های فعال یک استادیوم.
- **Index روی کلیدهای خارجی**: ایندکس‌گذاری کلیدهای خارجی **user_id** در **TRANSACTIONS_WALLET** و **RESERVATIONS** جهت افزایش سرعت JOINها.
- **Unique Index روی SEATS_RESERVED**: تضمین قطعی سطح دیتابیس برای جلوگیری از رزرو همزمان یک صندلی در حالت رزرو موقت.

۴. فاز سوم: توسعه سرویس بک‌اند (FastAPI)، Redis و امنیت

فاز سوم شامل توسعه سرویس بک‌اند ناهمگام (Async) با پریمورک **FastAPI** است [cite: 5]. طبق الزامات پروژه، از ORM استفاده نشده و تمام ارتباطات دیتابیس با Raw SQL ناهمگام (AsyncPG) به همراه Connection Pooling مدیریت می‌شوند [cite: 5].

۴.۱. معماری ماژولار پروژه (core/app)

- `core/config.py`: مدیریت مرکزی تنظیمات پروژه با [cite: 5] Pydantic Settings.
- `core/database.py`: مدیریت پادمان و Connection Pool ناهمگام PostgreSQL [cite: 5].
- `core/client_redis.py` & `utils_cache.py`: اتصال به Upstash Redis، اعمال کلیدهای کش و توابع [cite: 5] Cache Invalidation.
- `core/security.py` & `dependencies.py`: هشینگ Bcrypt، مدیریت توکن‌های JWT و تزریق وابستگی‌های [cite: 5] RBAC.
- `sync_tickets.py`: همگام‌سازی ناهمگام داده‌های بلیت در [cite: 5] Elasticsearch.

۴.۲. کاتالوگ API‌های توسعه‌یافته

ماژول / مسیر	متد	عنوان API	نحوه عملکرد و تکنولوژی‌های درگیر
<code>`auth/otp/send/`</code>	POST	ارسال کد OTP	تولید کد ۵ رقمی تصادفی و ذخیره در Redis با TTL برابر ۵ دقیقه [cite: 5].
<code>`auth/otp/verify/`</code>	POST	تأیید کد OTP	مقایسه کد ارسالی کاربر با مقدار ذخیره‌شده در Redis [cite: 5].
<code>`auth/signup/`</code>	POST	ثبت‌نام کاربر	هش رمز عبور با Bcrypt، درج در PostgreSQL و صدور

ماژول / مسیر	متد	عنوان API	نحوه عملکرد و تکنولوژی‌های درگیر
			.JWT[cite: 5]
`auth/login/`	POST	ورود به سیستم	اعتبارسنجی نام کاربری/رمز و صدور JWT Bearer Token[cite: 5]
`users/me/`	GET	دریافت پروفایل	خواندن از کش Redis (Cache-Aside)؛ در صورت Cache Miss، استعلام از دیتابیس[cite: 5]
`users/me/`	PUT	ویرایش پروفایل	بروزرسانی دیتابیس و پاکسازی صریح (Invalidation) کش کاربر در Redis[cite: 5]
`venues/cities/`	GET	شهرهای دارای ورزشگاه	استعلام لیست شهرها همراه با کشینگ جهت کاهش بار دیتابیس[cite: 5]
`tickets/`	GET	جستجو و فیلتر بلیت	اجرای کوئری متنی و فیلتر چندبعدی روی شاخص Elasticsearch[cite: 5]
`tickets/{id}/`	GET	جزئیات بلیت	دریافت مشخصات کامل مسابقه، صندلی‌ها و ویژگی‌های اختصاصی ورزش[cite: 5]

ماژول / مسیر	متد	عنوان API	نحوه عملکرد و تکنولوژی‌های درگیر
`reservations/`	POST	رزرو موقت بلیت	ایجاد قفل موقت روی صندلی در Redis با TTL برابر ۱۰ دقیقه (`SETNX`) جهت جلوگیری از Overbooking[cite: 5].
`reservations/{id}/pay/`	POST	نهایی‌سازی پرداخت	بررسی قفل، کسر از کیف پول یا درگاه، ثبت قطعی در PostgreSQL و سنک با Elasticsearch[cite: 5].
`reservations/me/`	GET	تاریخچه خریدها	فراخوانی Stored Procedure بلیت‌های خریداری‌شده کاربر[cite: 5].
cancellations/penalty-/` `check	GET	پیش‌نمایش جریمه	محاسبه درصد جریمه بر اساس زمان باقیمانده تا مسابقه[cite: 5].
`reservations/{id}/cancel/`	POST	ثبت کنسلی و استرداد	تغییر وضعیت رزرو، عودت وجه پس از کسر جریمه به کیف پول و آزادکننده صندلی[cite: 5].
`reports/`	POST	ثبت گزارش مشکل	ثبت اشکالات بلیت یا عدم تایید پرداخت جهت بررسی پشتیبانی[cite: 5].
`admin/reservations/flagged/`	GET	رزروهای مشکوک	استعلام رزروهای دارای رفتار مشکوک (مخصوص Admin/Support)[cite: 5].

۴.۳. امنیت و کنترل دسترسی (RBAC)

احراز هویت بر پایه ****JWT**** با الگوریتم `HMAC-SHA256` پیاده‌سازی شده است [cite: 5]. کنترل سطح دسترسی در فایل `dependencies.py` با متد `roles_check` انجام می‌شود که نقش‌های کاربر (`Customer`, `Support`, `Admin`) را ارزیابی کرده و در صورت عدم داشتن دسترسی لازم، خطای `HTTP 403 Forbidden` صادر می‌کند [cite: 5].

۵. فاز چهارم: رابط کاربری (Frontend) و Elasticsearch

در فاز چهارم، یک رابط کاربری سبک، سریع و کاملاً `Responsive` با ****Vanilla JavaScript**** و `HTML5`، `ES6+` و `CSS3` توسعه یافت تا بدون نیاز به فریم‌ورک‌های سنگین و فرآیندهای پیچیده `Build`، بالاترین کارایی را ارائه دهد [cite: 6].

۵.۱. ساختار فایل‌های فرانت‌اند

نام فایل	نوع / دسته	شرح وظیفه و نحوه عملکرد در پروژه
<code>`js/api.js`</code>	JavaScript Module	ماژول متمرکز شبکه: مدیریت <code>Fetch</code> ، افزودن اتوماتیک توکن <code>JWT</code> ، هدرها و پارس هوشمند خطاهای <code>Pydantic/FastAPI</code> [cite: 6].
<code>index.html` /` `search.js`</code>	HTML / JS	صفحه اصلی و جستجو: ارسال فیلترهای ورودی کاربر به <code>Elasticsearch</code> و رندر دینامیک کارت‌های بلیت [cite: 6].
<code>login.html` /` `auth.js`</code>	HTML / JS	ورود با رمز عبور و احراز هویت دومرحله‌ای <code>OTP</code> از طریق <code>Redis</code> [cite: 6].
<code>signup.html` /` `signup.js`</code>	HTML / JS	ثبت‌نام کاربران جدید و هدایت اتوماتیک به صفحه ورود [cite: 6].
<code>checkout.html` /` `checkout.js`</code>	HTML / JS	تکمیل رزرو، قفل صندلی، درخواست رمز پویا کارت بانکی و پرداخت نهایی [cite: 6].

نام فایل	نوع / دسته	شرح وظیفه و نحوه عملکرد در پروژه
profile.html` /` ` `profile.js	HTML / JS	داشبورد کاربر: مشاهده بلیت‌ها، استعلام جریمه کنسلی، ثبت کنسلی، ویرایش پروفایل و ثبت گزارش مشکل[cite: 6].
`css/style.css`	CSS3 Style	استایل‌های عمومی: متغیرهای رنگی، Grid CSS، Flexbox، کارت‌های بلیت، مدال‌ها و Responsive Design[cite: 6].

۵.۲. تحلیل ماژول شبکه (`js/api.js`)

این فایل الگوی ****Singleton Client**** را پیاده‌سازی کرده است[cite: 6]. مهم‌ترین ویژگی آن، استخراج دقیق پیام‌های خطای اعتبارسنجی (Validation Errors) در FastAPI است که معمولاً به صورت آرایه‌ای در `detail` بازمی‌گردند[cite: 6]:

```
// مدیریت هوشمند خطاهای Pydantic / FastAPI در api.js
if (!response.ok) {
  let errorMessage = `خطای سرور (${response.status})`;
  if (typeof data.detail === 'string') {
    errorMessage = data.detail;
  } else if (Array.isArray(data.detail)) {
    errorMessage = data.detail.map(err => err.msg || 'خطای ورودی').join(' | ');
  } else if (data.message) {
    errorMessage = data.message;
  }
  throw new Error(errorMessage);
}
```

۵.۳. یکپارچه‌سازی Elasticsearch 8.x

برای از بین بردن بار کوئری‌های سنگین متنی از روی دیتابیس PostgreSQL اصلی، Elasticsearch به عنوان موتور جستجو استفاده شد[cite: 6]:

- **ایندکس‌گذاری خودکار (Pipeline Indexing):** هنگام ثبت یا ویرایش مسابقه در PostgreSQL، داده‌های متنی (نام تیم‌ها، ورزشگاه، شهر و نوع ورزش) به Document در Elasticsearch تبدیل می‌شوند[cite: 6].
- **کوئری‌های ترکیبی (Multi-Match & Range Filters):** اجرای همزمان جستجوی متنی روی چند فیلد و اعمال فیلترهای دامنه قیمت و تاریخ با سرعت پاسخ‌دهی زیر چند

میلی‌ثانیه [cite: 6].

- **Offloading دیتابیس:** حفظ کارایی دیتابیس PostgreSQL برای عملیات حیاتی مانند پردازش پرداخت و رزرو [cite: 6].

۶. تحلیل جریان‌های کاری کلیدی (End-to-End Workflows)

سناریوی ۱: جستجو، انتخاب صندلی و رزرو موقت

1. کاربر نام تیم، شهر یا ورزش را در `index.html` وارد می‌کند [cite: 6].
2. فرانت‌اند درخواست را به متد `tickets.search` در `api.js` می‌فرستد که مستقیماً روی Elasticsearch اجرا می‌شود [cite: 6].
3. با انتخاب صندلی، بک‌اند با دستور `SETNX` صندلی را به مدت ۱۰ دقیقه در Redis قفل می‌کند تا فرد دیگری نتواند آن را انتخاب کند (جلوگیری از Overbooking) [cite: 5, 6].

سناریوی ۲: نهایی‌سازی پرداخت و صدور بلیت

1. کاربر به صفحه `checkout.html` منتقل شده و شماره کارت را وارد می‌کند [cite: 6].
2. با کلیک روی "درخواست رمز پویا"، کد یکبارمصرف شبیه‌سازی‌شده صادر می‌شود [cite: 6].
3. پس از تایید پرداخت، تراکنش مالی ثبت شده، وضعیت صندلی در PostgreSQL قطعی می‌گردد و کلید Redis پاکسازی می‌شود [cite: 5, 6].

سناریوی ۳: استعلام جریمه کنسلی و عودت وجه به کیف پول

1. کاربر در داشبورد خود (`profile.html`) روی بلیت خریداری‌شده دکمه استعلام کنسلی را کلیک می‌کند [cite: 6].
2. API مسیر `cancellations/penalty-check/` فاصله زمانی تا بازی را محاسبه کرده و درصد جریمه را نشان می‌دهد [cite: 5, 6].

3. پس از تایید کاربر، مبلغ پس از کسر جریمه به کیف پول عودت داده شده، صندلی در دیتابیس آزاد می‌شود و شاخص Elasticsearch بروز می‌گردد[cite: 5, 6].

۷. ارزیابی نهایی و دستاوردهای پروژه

پروژه **BookingSportTickets** با موفقیت تمامی اهداف آموزشی و فنی را محقق ساخت. ترکیبی از معماری دیتابیس نرمال‌سازی‌شده (3NF)، سرعت بالای FastAPI و Raw SQL، قابلیت مدیریت همزمانی با Redis، قدرت جستجوی Elasticsearch و چابکی فرانت‌اند Vanilla JS باعث گردید محصولی کامل، ایمن و آماده استقرار ارائه شود.

خلاصه دستاوردهای کلیدی پروژه:

- طراحی ۳۰ جدول دیتابیس طبق استاندارد 3NF و الگوی Polymorphism برای رشته‌های ورزشی مختلف[cite: 4].
- کاهش بار دیتابیس اصلی با بهره‌گیری از Cache-Aside در Redis و موتور Elasticsearch[cite: 5, 6].
- حل کامل مشکل Overbooking با پیاده‌سازی قفل موقت ۱۰ دقیقه‌ای صندلی‌ها در Redis[cite: 5].
- توسعه فرانت‌اند بدون وابستگی به فریم‌ورک‌های سنگین با زمان بارگذاری آنی و مدیریت متمرکز [cite: 6].API

گزارش نهایی پروژه درس مدیریت پایگاه داده و توسعه بک‌اند | تهیه شده توسط تیم سه نفره دانشجویی |

مرداد ۱۴۰۵